

Organizacja Pracy i Architektura Projektu PZ-2026

1. Struktura Repozytorium (Monorepo)

Aplikacja jest podzielona na dwa główne katalogi w jednym repozytorium, aby zapewnić separację logiki biznesowej od interfejsu użytkownika:

- /frontend – aplikacja kliencka (React + Tailwind CSS).
- /backend – serwer API oraz logika kryptograficzna/steganograficzna (Python + FastAPI).

2. Backend (Python / FastAPI)

- **Technologia:** Python + FastAPI.
- **Uruchamianie serwera:** Serwer uvicorn nasłuchujący na porcie **3000** (np. uvicorn index:app --port 3000 --reload).
- **Struktura plików:** * Głównym plikiem wejściowym jest index.py, który agreguje wszystkie endpointy.
 - Każdy algorytm ma swój dedykowany plik nazwany od jego nazwy własnej (np. szyfr_atbash.py, szyfr_cezara.py), co ułatwi pracę równoległą i zminimalizuje konflikty w Git.
- **Komunikacja z API:**
 - **WAŻNE:** Do komunikacji używamy zapytań **POST** (nie GET). Wynika to z faktu przesyłania dużych tekstów oraz plików multimedialnych (BMP/WAV), które muszą być umieszczone w ciele zapytania (Body/FormData).
- **Zależności:** Wszystkie pakiety pythona (fastapi, uvicorn, ew. biblioteki do obsługi wav/bmp) muszą być na bieżąco dodawane do pliku requirements.txt.

3. Frontend (React / Vite)

- **Technologia:** React.js uruchamiany przez Vite (domyślny port **5173**).
- **Język i stylowanie:** Czysty JavaScript (bez TypeScripta, aby przyspieszyć development i uniknąć problemów z typowaniem dynamicznych odpowiedzi) oraz Tailwind CSS do szybkiego budowania interfejsu.

- **Zarządzanie pakietami:** npm lub yarn (do ustalenia). Przed rozpoczęciem pracy pobieramy pakiety komendą npm install/yarn install.

4. System Kontroli Wersji (Git)

Pracujemy w oparciu o system odgałęzień (branchy), chroniąc główny branch main.

- **Tworzenie branchy:** Każde zadanie (implementacja algorytmu, poprawka błędu) realizowane jest na osobnym branchu odbitym od main.
- **Konwencja nazewnictwa:** [nazwa_tasku]_[nazwisko1]_[nazwisko2].
 - *Przykład:* szyfr_cezara_nowak_kowalski
 - Dzięki temu od razu widać, kto odpowiada za dany kod i ewentualne jego poprawki.
- **Zasady łączenia kodu (Merge/Pull Request):**
 - Przed próbą połączenia swojego kodu z main, **obowiązkowo** pobieramy najnowsze zmiany z main do swojego brancha (git pull origin main lub git merge main).
 - Osoby przypisane do brancha samodzielnie rozwiązują ewentualne konflikty u siebie lokalnie.
 - Upewniamy się, że aplikacja po rozwiązaniu konfliktów nadal działa minimalizując możliwość powstania błędów na main.
 - Tworzymy Merge/Pull Request do main.

5. Podział Zadań i Praca Modułowa (Kryptografia)

Zgodnie z wymaganiami, poszczególne osoby odpowiadają za swoje pliki szyfrów. Funkcja w pliku (np. szyfr_cezara.py) powinna przyjmować ciąg znaków (oraz ew. klucz) i zwracać zaszyfrowany/odszyfrowany tekst, nie martwiąc się o logikę API. Podpięciem tego pod endpoint w index.py zajmuje się osoba integrująca dany moduł.